COS80023 Big Data – Lab 9

Lab 9: Pass Task 9 – Natural Language Processing

Student Name: Arun Ragavendhar Arunachalam Palaniyappan

Student ID: 104837257

Exercise 1

Q1. What do these numbers tell you? Why did we give table() the argument data.raw$type?

```
> View(data.raw)
> # Makes the type field (ham or spam) a factor:
> data.raw$type <- as.factor(data.raw$type)
> # Shows the proportions of the values of the type attribute:
> prop.table(table(data.raw$type))

 ham spam
0.86 0.14
> 
```

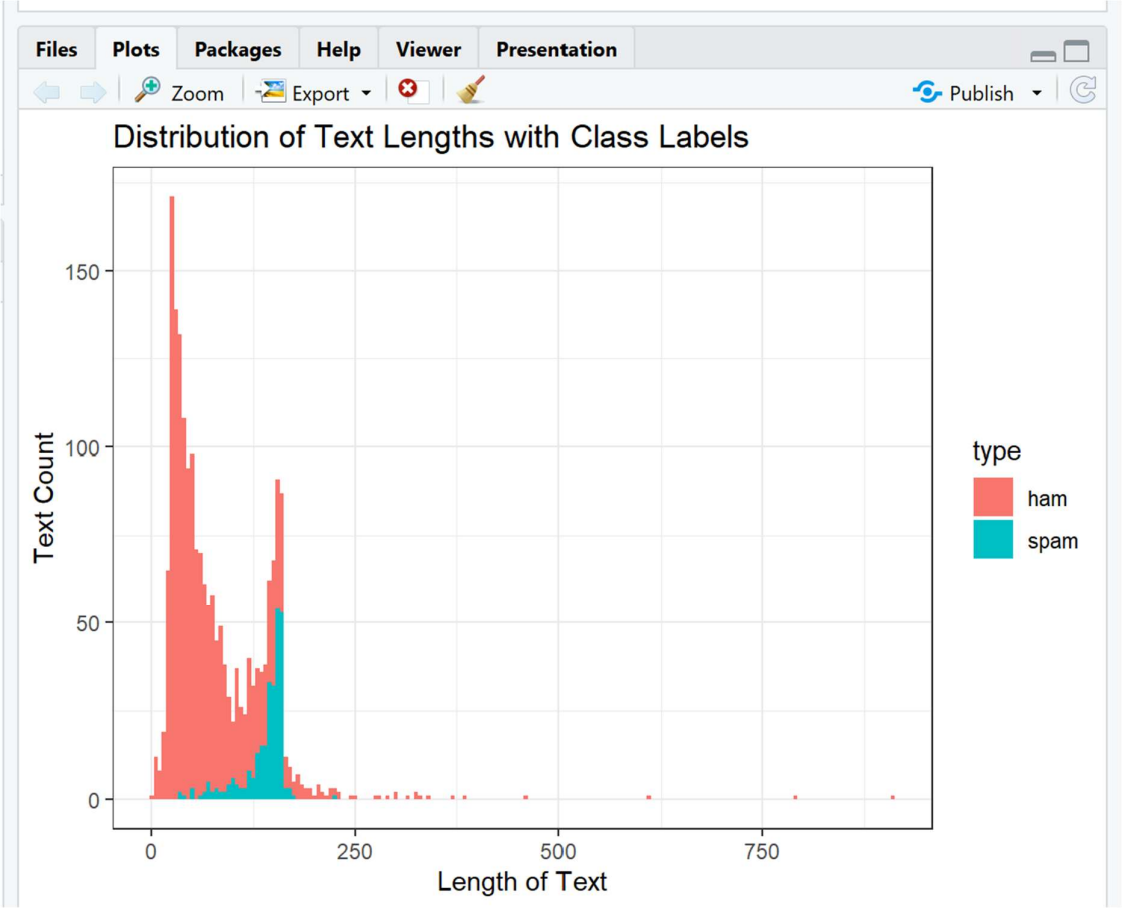
The command table(data.raw$type) counts how many SMS messages are labelled ham and how many are spam.
prop.table() then converts these counts into proportions.
These numbers show that most messages in the dataset are ham (86%) while only a smaller share is spam (14%). We gave table() function, the argument data.raw$type, because that column holds the message type label (ham or spam). Looking at these proportions tells us the dataset is imbalanced, with far more legitimate messages than spam, which is typical in real SMS collections.

Exercise 2

```
> #find the statistics for each type (ham and spam) separately:
> indexes <- which(data.raw$type=="ham")
> ham <- data.raw[indexes,]
> spam <- data.raw[-indexes,]
> spam$TextLength <- nchar(spam$text)
> ham$TextLength <- nchar(ham$text)
> summary(ham$TextLength)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
   2.00   33.00   53.00   72.29   96.00  910.00 
> summary(spam$TextLength)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
   33.0   132.8   148.0   139.2   156.0   223.0 
>
> #Plot the histogram
> ggplot(data.raw, aes(x = TextLength, fill = type)) +
+   theme_bw() +
+   geom_histogram(binwidth = 5) +
+   labs(y = "Text Count", x = "Length of Text",
+        title = "Distribution of Text Lengths with Class Labels")
> 
```

Q2. What does this code do, and what does the output tell us about ham and spam?

The code separates the data into ham and spam groups, measures the number of characters in each message using nchar(), and then plots a histogram of message lengths.
From the plot and summary values we can see that ham messages are generally shorter and more consistent, while spam messages are usually longer and vary more in length because they contain promotions, URLs, or phone numbers.
This difference in message length is an early indicator that text-length features can help distinguish spam from ham.



In this histogram, the **Length of Text** axis measures how many characters each message contains, while the **Text Count** axis shows how many messages fall into each length range.

Exercise 3

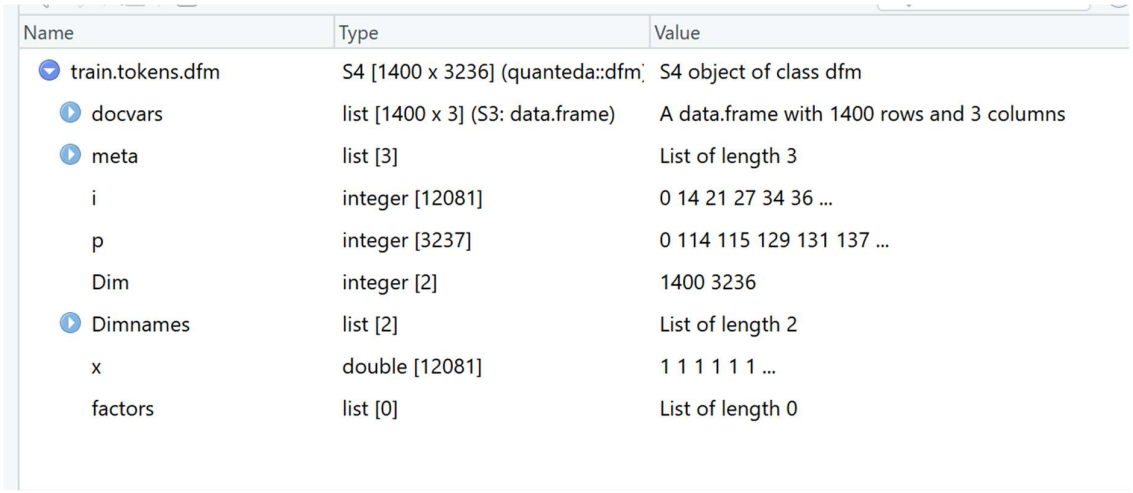
Q3. Which NLP processing steps are performed by this code? Is the outcome what you expect?

The code performs the main text-pre-processing steps used in Natural Language Processing:

1. **Tokenisation** – splits each message into individual words.
2. **Lower-casing** – makes all words lower case so “Buy” and “buy” are treated the same.
3. **Stop-word removal** – removes very common words such as the, and, is that add little meaning.
4. **Stemming** – reduces words to their root form (e.g., offers, offering → offer).

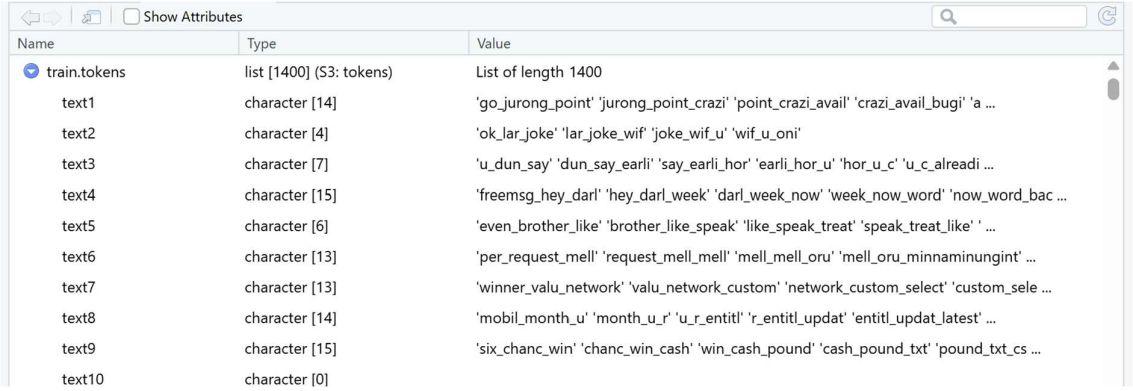
After these steps, the total number of unique tokens becomes smaller and the text cleaner and more uniform.
Yes, this is exactly the expected outcome before creating the document-frequency matrix for modelling.

The DFM was created



Name	Type	Value
train.tokens.dfm	S4 [1400 x 3236] (quanteda::dfm)	S4 object of class dfm
docvars	list [1400 x 3] (S3: data.frame)	A data.frame with 1400 rows and 3 columns
meta	list [3]	List of length 3
i	integer [12081]	0 14 21 27 34 36 ...
p	integer [3237]	0 114 115 129 131 137 ...
Dim	integer [2]	1400 3236
Dimnames	list [2]	List of length 2
x	double [12081]	1 1 1 1 1 1 ...
factors	list [0]	List of length 0

Q4. How do you create 3-grams? How does data.tokens change?



Show Attributes

Name	Type	Value
train.tokens	list [1400] (S3: tokens)	List of length 1400
text1	character [14]	'go_jurong_point' 'jurong_point_crazi' 'point_crazi_avail' 'crazi_avail_bugi' 'a ...
text2	character [4]	'ok_lar_joke' 'lar_joke_wif' 'joke_wif_u' 'wif_u_oni'
text3	character [7]	'u_dun_say' 'dun_say_earli' 'say_earli_hor' 'earli_hor_u' 'hor_u_c' 'u_c_alreadi ...
text4	character [15]	'freemsg_hey_darl' 'hey_darl_week' 'darl_week_now' 'week_now_word' 'now_word_bac ...
text5	character [6]	'even_brother_like' 'brother_like_speak' 'like_speak_treat' 'speak_treat_like' ' ...
text6	character [13]	'per_request_mell' 'request_mell_mell' 'mell_mell_oru' 'mell_oru_minnaminungint' ...
text7	character [13]	'winner_valu_network' 'valu_network_custom' 'network_custom_select' 'custom_sele ...
text8	character [14]	'mobil_month_u' 'month_u_r' 'u_r_entitl' 'r_entitl_updat' 'entitl_updat_latest' ...
text9	character [15]	'six_chanc_win' 'chanc_win_cash' 'win_cash_pound' 'cash_pound_txt' 'pound_txt_cs ...
text10	character [0]	

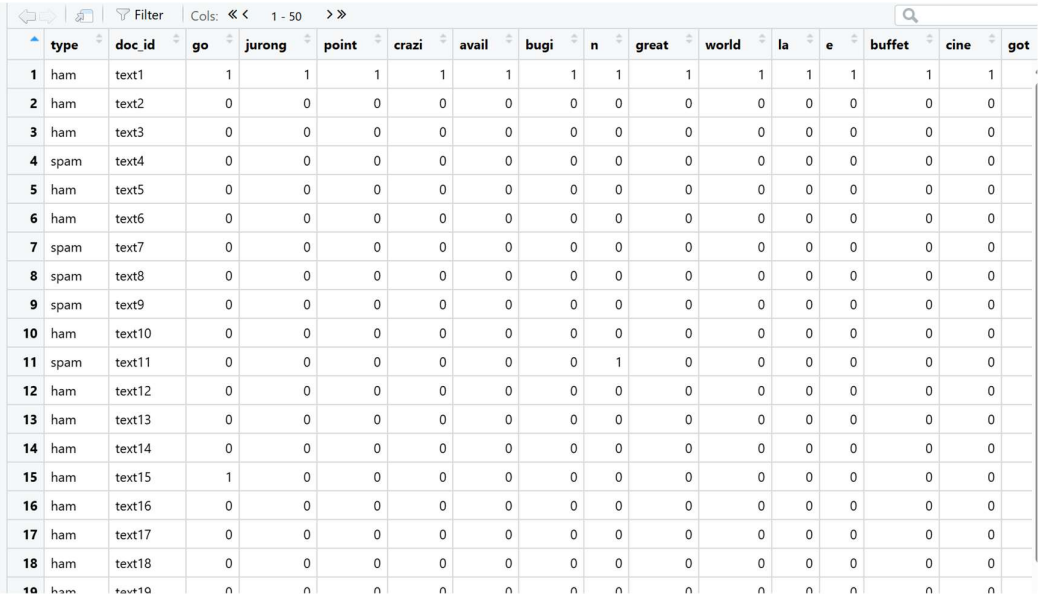
In recent quanteda versions, the ngrams= argument on tokens() is **ignored/deprecated**. The reliable way in quanteda is to first create normal tokens (by default 1), then turn them into **3-grams** using tokens_ngrams().

```
data.tokens <- tokens_ngrams(data.tokens, n = 3)
```

Exercise 4

Q5. What has cbind changed in the data?

cbind(type = train.raw$type, data.tokens.df) adds the **class label column** (ham or spam) to the table of token features.
Before cbind, the dataset only contained the input variables (word-frequency counts).
After cbind, every row now has both its features and its correct output label.
This combined table is what the machine-learning algorithm uses for training.



Filter | Cols: « < 1 - 50 > »

	type	doc_id	go	jurong	point	crazi	avail	bugi	n	great	world	la	e	buffet	cine	got
1	ham	text1	1	1	1	1	1	1	1	1	1	1	1	1	1	
2	ham	text2	0	0	0	0	0	0	0	0	0	0	0	0	0	
3	ham	text3	0	0	0	0	0	0	0	0	0	0	0	0	0	
4	spam	text4	0	0	0	0	0	0	0	0	0	0	0	0	0	
5	ham	text5	0	0	0	0	0	0	0	0	0	0	0	0	0	
6	ham	text6	0	0	0	0	0	0	0	0	0	0	0	0	0	
7	spam	text7	0	0	0	0	0	0	0	0	0	0	0	0	0	
8	spam	text8	0	0	0	0	0	0	0	0	0	0	0	0	0	
9	spam	text9	0	0	0	0	0	0	0	0	0	0	0	0	0	
10	ham	text10	0	0	0	0	0	0	0	0	0	0	0	0	0	
11	spam	text11	0	0	0	0	0	0	1	0	0	0	0	0	0	
12	ham	text12	0	0	0	0	0	0	0	0	0	0	0	0	0	
13	ham	text13	0	0	0	0	0	0	0	0	0	0	0	0	0	
14	ham	text14	0	0	0	0	0	0	0	0	0	0	0	0	0	
15	ham	text15	1	0	0	0	0	0	0	0	0	0	0	0	0	
16	ham	text16	0	0	0	0	0	0	0	0	0	0	0	0	0	
17	ham	text17	0	0	0	0	0	0	0	0	0	0	0	0	0	
18	ham	text18	0	0	0	0	0	0	0	0	0	0	0	0	0	
19	ham	text19	0	0	0	0	0	0	0	0	0	0	0	0	0	

Q6. Why do we need to align the test set columns to the training set columns?

When we build the document-frequency matrix (DFM) for the test set, it may contain different tokens than the training set.
If we try to predict using mismatched columns, the model will not understand the extra or missing words.
By running

```
dfm_select(test.tokens.dfm, pattern = train.tokens.dfm,
selection = "keep")
```

we keep only the token columns that also exist in the training data, so both data frames have exactly the same structure.

This alignment ensures the model reads each column as the same word or 3-gram in both training and testing phases.

Q7. How does the model perform on the training and testing sets (accuracy, specificity, sensitivity)?

```
> confusionMatrix(table(trainresult, train.tokens.df$type))
Confusion Matrix and Statistics


trainresult  ham spam
       ham  1149   49
       spam   55  147

               Accuracy : 0.9257
                 95% CI : (0.9107, 0.9389)
    No Information Rate : 0.86
    P-Value [Acc > NIR] : 1.198e-14

                  Kappa : 0.6954

 Mcnemar's Test P-Value : 0.6239

            Sensitivity : 0.9543
            Specificity : 0.7500
         Pos Pred Value : 0.9591
         Neg Pred Value : 0.7277
             Prevalence : 0.8600
         Detection Rate : 0.8207
   Detection Prevalence : 0.8557
      Balanced Accuracy : 0.8522

       'Positive' Class : ham
```

Model performance on training data

```
Confusion Matrix and Statistics


testresult ham spam
      ham  479   27
      spam  37   57

               Accuracy : 0.8933
                 95% CI : (0.8658, 0.9169)
    No Information Rate : 0.86
    P-Value [Acc > NIR] : 0.009078

                  Kappa : 0.5781

 Mcnemar's Test P-Value : 0.260589

            Sensitivity : 0.9283
            Specificity : 0.6786
         Pos Pred Value : 0.9466
         Neg Pred Value : 0.6064
             Prevalence : 0.8600
         Detection Rate : 0.7983
   Detection Prevalence : 0.8433
      Balanced Accuracy : 0.8034

       'Positive' Class : ham
```

Model Performance on testing data

The model performs well overall, showing slightly better results on the training data than on the testing data.

On the training set, it achieved an accuracy of 92.6%, with sensitivity = 0.954 and specificity = 0.750. This means it correctly identified most ham messages and recognised many spam messages. The Kappa value of 0.695 indicates good agreement between predicted and actual classes.

On the testing set, the accuracy was 89.3%, with sensitivity = 0.928 and specificity = 0.679. These values are slightly lower, as expected for unseen data, and the Kappa value of 0.578 still shows moderate to good agreement.

The difference between training and testing results is small, showing that the model is not overfitted and can generalise well to unseen data. Also, the confusion matrix shows that while the model is very good at identifying ham messages, it sometimes misses a noticeable number of spam messages. This means it is reliable for classifying normal messages correctly, but could be improved further to catch more spam, perhaps by adjusting the threshold or using techniques to handle the class imbalance.